

WebShield

detectar · decidir · bloquear · registrar · visualizar



Detección con ML, bloqueo en línea y visibilidad



WAF Proxy

FastAPI + nginx



ML API

FastAPI + scikit-learn



App Simulada

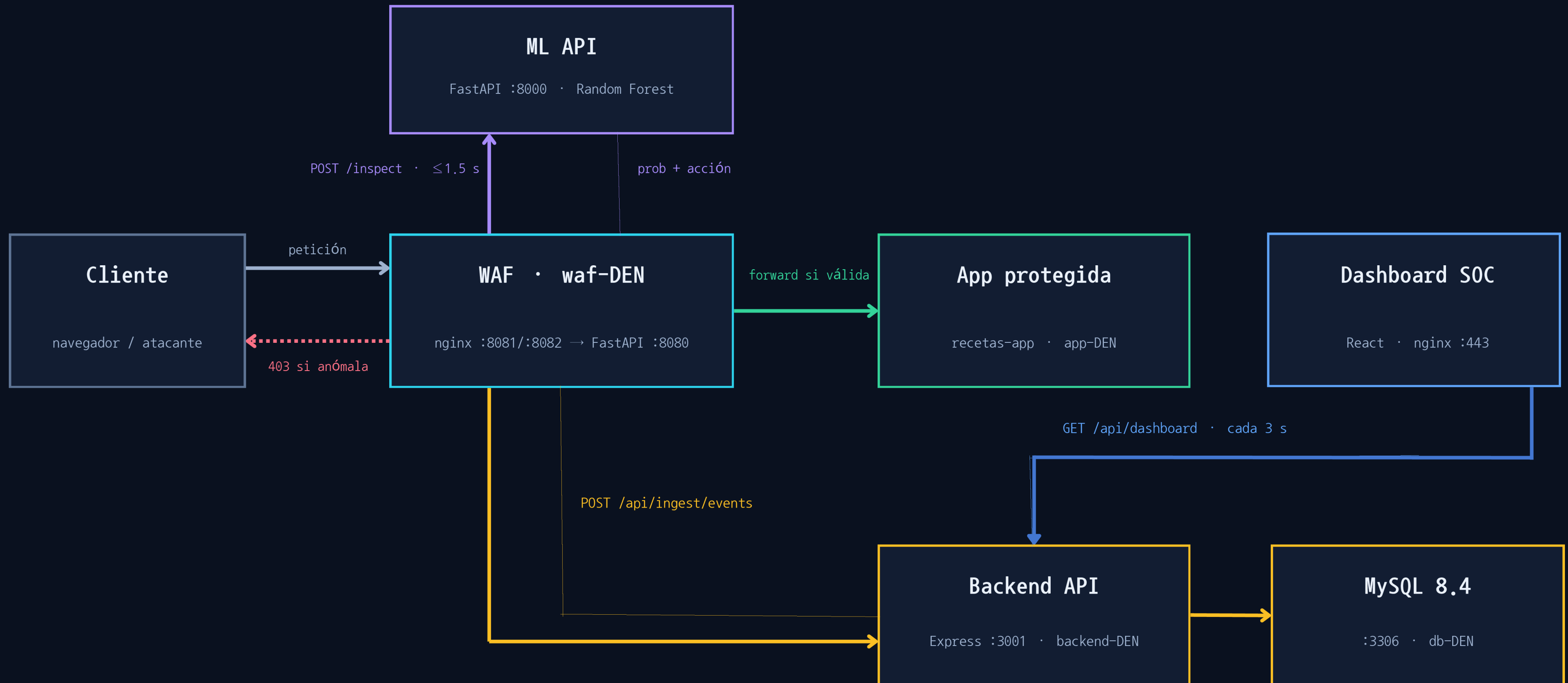
Express + MySQL



Dashboard SOC

React + Tailwind

El viaje de una petición, de extremo a extremo



Del dataset público al modelo en producción

DATASET

ECML/PKDD 2007 – Web Attacks

- Peticiones HTTP
- Campos por petición (URI, Método, Query, Body)
- Limpieza
- Split 80 / 20 entrenamiento / prueba

```
RandomForestClassifier(n_estimators=100)
```

scikit-learn 1.6.1 · export joblib (5.9 MB)

1

Ingeniería de features

Cada petición se convierte en un vector de 42 señales numéricas

2

Entrenamiento

Random Forest con 100 árboles

3

Export versionado

Modelo + metricas + fecha.

4

Servido en producción

La ML API carga el artefacto y lo recarga si el archivo cambia

Features de cada petición HTTP

42

features por petición

19 Estructura y léxico

Longitudes, caracteres especiales,
keywords SQL/XSS/traversal

15 Reglas OWASP CRS

RCE, LFI/RFI, SQLi, XSS,
Log4Shell, NoSQL, null bytes

5 Fingerprinting de User-Agent

Scanners conocidos (sqlmap)
vs navegadores reales

3 Categóricas del protocolo

Metodo HTTP (POST, PUT) y
version del protocolo (HTTP/1.1)

Mejora medible sobre el baseline publicado

90.75 %

Accuracy — exactitud global en el set de test

0.9584

ROC-AUC — separación entre tráfico válido y anómalo

0.9745

PR-AUC — precisión sobre la clase de ataque

ACCURACY EN TEST (%)

Baseline · 22 features

89.58

WebShield · 42 features

90.75

80

85

90

95

ANOMALY_THRESHOLD

umbral configurable

metricas en .joblib

trazabilidad por version

Cada petición se decide en milisegundos

1

Entrada por nginx

puertos 8081 (dashboard) y 8082 (app pública); inyecta X-WebShield-App para enrutar

2

Extracción

método, URI, query, headers, IP de origen y body (límite 1 MB)

3

Consulta al modelo

POST /inspect a la ML API, con timeout de 1.5 s

4

Decisión

$\text{prob} \geq \text{umbral} \rightarrow \text{HTTP 403}$ · $\text{prob} < \text{umbral} \rightarrow \text{forward transparente}$

5

Registro asíncrono

el evento se ingesta fire-and-forget: el cliente nunca espera



respuesta del WAF

```
$ curl 'https://waf:8082/recetas?id=1 UNION...'
```

HTTP/1.1 403 Forbidden

```
{
  "detail": "Request blocked by WebShield",
  "label": "anomalous",
  "prob_anomalous": 0.92,
  "model_version": "1.0.0"
}
```

FAIL_MODE open | closed

open | closed

Idempotencia por event_id

los reintentos del WAF nunca duplican eventos

Visibilidad en tiempo real para el analista

- Tabla en vivo
- Filtros y semáforo de score
- Drawer de detalle por event
- Inspección manual de peticiones



i18n ES / EN

persistido por usuario



Claro / oscuro

respetar el tema del SO



Accesible

ARIA, teclado, reduced motion



React 19 + Vite

Tailwind 4 · cookies JWT

webshield — live events

● live

refresh: 3 s

filter: all

sesión: cookie JWT

TIME	SOURCE IP	MET	URI	VERDICT	SCORE	ACTION
12:01:14	45.155.10.88	GET	/recetas?id=1' UNION SE...	anomalous	0.92	BLOCKED
12:01:11	172.16.67.20	GET	/api/recetas	valid	0.04	ALLOWED
12:01:03	45.155.10.88	POST	/login.php	anomalous	0.88	BLOCKED
12:00:58	172.16.67.31	GET	/categorias	valid	0.07	ALLOWED
12:00:49	91.240.118.7	GET	/.env	anomalous	0.83	BLOCKED

click en una fila → drawer con headers y body completos

Instancias

VLAN infra2Red · 172.16.67.0/24 · OpenStack — sin IPs públicas

waf-DEN .172

nginx :8081/:8082 + uvicorn :8080 —
único punto expuesto

ML (físico) .67

FastAPI :8000 — sirve el Random
Forest, solo visible al WAF

app-DEN .149

app víctima (recetas-app) — solo
recibe tráfico del WAF

frontend-DEN .148

nginx :443 — termina TLS, sirve
React y proxy /api

backend-DEN .144

Node 22 · Express :3001 — servicio
systemd webshield-api

db-DEN .136

MySQL 8.4 :3306 — solo acepta
conexiones del backend



Acceso externo

Tunel SSH via bastion
solo :8082 del WAF expuesto



Mínimo privilegio

Security Groups por puerto
tokens rotados en pareja

Defensa



Red

VLAN privada sin IPs públicas; Security Groups por puerto; MySQL solo visible para el backend; operación únicamente por bastión SSH.



Transporte

TLS en el frontend (nginx :443) y en el backend; cookies marcadas Secure



Aplicación

Content-Security-Policy (recursos que puede cargar), rate limiting de 10 req / 15 min en login y validación de todo payload de ingesta.



Credenciales

Contraseñas con bcrypt, sesión JWT en cookie HttpOnly SameSite=Strict y tokens Bearer

Anatomía de un ataque bloqueado

ATAQUE ▶ **GET /recetas?id=1' UNION SELECT email, password_hash FROM usuarios--**

01 · Features

```
has_sql_advanced = 1  
cnt_quote = 1  
cnt_comment = 1
```

el WAF convierte la petición en su vector de 42 señales

02 · Inferencia

```
prob_anomalous = 0.92  
action = block
```

el Random Forest responde en milisegundos

03 · Bloqueo

```
HTTP/1.1 403  
Forbidden
```

la app protegida nunca llega a ver la petición

04 · Visibilidad

```
evento en dashboard  
en ≤ 3 s
```

verdict, score, headers y payload completos para el analista

Demo en vivo: lanzamos este ataque contra la app protegida y lo vemos aparecer bloqueado en el dashboard, junto al tráfico legítimo que sí pasa.

WebShield

detectar · decidir · bloquear · registrar · visualizar

